

Course Code: CSE 499A

Section: 15

Project Group No: Group 06

Group Members:

Md.Murad Ul Momin (ID: 2232360642)

Rima Saha (ID: 2132517042)

Sakia Tabassum Anonna (ID: 2221433042)

Ashraful Hossain Riyan (ID: 2232845642)

PRIVATE SHIELD: Week 1 Progress Report

I. INTRODUCTION

The PRIVATE SHIELD project delivers a Privacy-Preserving Federated Intelligence System for IoT Threat Detection. This report summarizes the Week 1 milestones, establishing the baseline framework encompassing the local telemetry simulation engine, data preprocessing foundations, defensive security sandboxes, and web infrastructure.

II. SYSTEM IMPLEMENTATION

Development was divided into four core engineering vectors to create a cohesive local environment pipeline:

A. IoT Simulator & Telemetry Pipeline

An enterprise-grade message queuing system was built using the Eclipse Mosquitto MQTT broker platform. A runtime script (`fake_device.py`) was implemented using the `paho-mqtt` dependency to simulate edge-level hardware footprints. The script parses execution arguments dynamically to enable multi-device instances. It broadcasts continuous data packets over localhost loopbacks (127.0.0.1:1883) at a strict 3-second polling interval. Generated payload properties comprise isolated telemetry points such as device identifiers, random temperatures (20.0°C–35.0°C), operational humidity levels (40.0%–80.0%), and packet metadata. Pipeline stability was verified by simultaneously executing distinct `cam_01`, `sensor_02`, and `plug_03` edge instances.

B. ML Environment & Data Engineering

To facilitate analytical parsing, the structural format of benign network datasets was thoroughly evaluated. The pipeline checked for data integrity anomalies and cleaned the dataset by removing exactly 9,153 duplicate rows to ensure higher training quality.

The analytical setup established step-by-step programming logic designed to combine distinct data blocks, shuffle raw profiles to mitigate ordering biases, normalize numerical ranges via scaling, and partition the final vectors into a balanced 80% training set and a 20% validation testing set. Rather than relying on a local machine layer, development shifted exclusively to the cloud-based Kaggle ecosystem. This circumvented downloading large raw data files, bypassed complex local framework installations, and leveraged free cloud processing units.

C. Threat Assessment Sandbox

A sandboxed network attack environment was isolated inside an Oracle VirtualBox architecture by deploying a pre-built Kali Linux VM appliance with bridged host-only adapter interfaces. Automated assessment packages (`nmap` and `hping3`) were successfully audited to verify baseline readiness.

A custom multi-threaded socket framework script (`attack_dos.py`) was built on the host machine to execute parallelized target connection

flooding. The module spawns 100 concurrent threads to establish rapid, brief TCP connections targeting the port 1883 ingress. Real-time measurement metrics log connection flood velocity attempts per second to accurately assess system capacity bounds under peak volumetric saturation.

D. Web Framework Integration

The client-facing UI and backend interface scaffolds were established. The client layout was scaffolded using the Node.js runtime environment through a modular React application architecture (`shield-web-dashboard`).

Concurrently, the API backend foundation was deployed using FastAPI paired with the Uvicorn high-performance ASGI web server. A micro-REST liveness verification route (`GET /ping`) was successfully written and served on port 8000. System operational tests confirmed perfect end-to-end responses, with the browser querying tool successfully receiving the structured validation payload `{"status":"ok"}`.

III. VERIFICATION RESULTS

Comprehensive local testing confirmed successful operation:

- **Telemetry Loop:** The MQTT broker successfully accepted concurrent data entries across multiple ports from three simultaneously running client profiles.
- **Network Saturation:** The DoS tool validated continuous throughput under load without causing process blocks or memory exhaustion.
- **Liveness Probe:** Web services displayed instant hot-reload routing connectivity with valid JSON responses.

IV. WEEK 2 OBJECTIVES

Moving into Week 2, the primary goal shifts toward handling active MQTT streams and generating initial datasets. The key milestones include:

1. Upgrading the IoT simulator to support three diverse real-world behavior profiles: high-bandwidth cameras, low-frequency environment sensors, and smart plugs.
2. Writing a continuous data logging utility (`mqtt_logger.py`) to subscribe to active device topics, apply timestamps, auto-label regular baseline traffic, and commit entries to an internal storage matrix (`normal.csv`).
3. Constructing an advanced automated simulator capable of switching between three distinct attack methodologies: Denial of Service (DoS) floods, progressive port scanning loops, and synchronized Mirai botnet surges.
4. Designing a command-line interface monitor that pulls live shared counter files to dynamically reflect active device numbers, transmission frequency rates, and security alert logs.